SAGE: Student-focused Adaptive Guidance Engine for Intelligent Tutoring with Regulated Learning

Mahady H Rayhan, Gabriel R. Edwards, Ashish Pandey, Vani Seth, Nilesh Salvi,
Noah Glaser, Marisa Chrysochoou, Prasad Calyam
University of Missouri - Columbia, USA
Email: {mhr6wb, gre4bt, apfd6, vs4ky, salvin, noah.glaser, mczhd, calyamp}@missouri.edu

***Abstract*—Public Large Language Models (LLMs) are being increasingly explored for aiding students to learn difficult concepts in STEM education. However, public LLM-based chatbots do not provide course-regulated answers that are important in a tutoring context, where instructors do not want students to work with chatbots on out-of-scope material, seek answers that hinder learning, or to bypass curricular sequencing. In this paper, we present "SAGE" (Student-focused Adaptive Guidance Engine) as part of an intelligent tutoring framework that can enforce instructor-regulated learning through goal-oriented and learning state based content access controls to ensure scaffolded, curricular sequencing and step-by-step guidance of students. SAGE access control features a three-layer gatekeeper mechanism that combines role-based, attribute-based, and cognitive state–based controls built on top of a knowledge graph of learner state and related course content. We evaluate SAGE in an exemplar programming course scenario and compare its performance against state-of-the-art LLM baselines using a custom educational benchmark. Experimental results on the benchmark show that prompt-based tutors (42%) perform similar to SAGE (44%) on metrics of helpfulness owing to prompt engineering. However, they frequently violate instructor policy boundaries, with SAGE providing 90% security for curriculum access control policies compared to 30% for prompt-based tutors.**

***Index Terms*—Human-AI Teaming, Intelligent Tutoring, Knowledge Graphs, Cognitive Scaffolding, Access Control**

I. Introduction

The rapid proliferation of LLMs has transformed the landscape of Intelligent Tutoring Systems (ITS). Unlike earlier rule-based or scripted tutors, public LLMs such as ChatGPT can explain concepts, debug code, and synthesize examples in real time, creating the promise of scalable, personalized instruction for millions of learners [1]. At the same time, their widespread use in academic settings has raised unprecedented concerns about academic integrity, over-reliance on automation, and the hindrance to genuine learning habits/processes [1] [2].

At the heart of this tension is a misalignment between the objectives of general-purpose LLMs and the goals of education, particularly in STEM area course curriculum. LLMs are optimized to be helpful, i.e., to produce fluent, direct and complete answers to user prompts. Effective instruction, however, seeks to cultivate initially Instructor-Regulated Learning (IRL), where instructors actively plan, monitor and assess student learning. However, ultimately instruction could serve students' Self-Regulated Learning (SRL), in which learners actively plan, monitor, and reflect on their own learning. Zimmerman's Cyclical Model [3] characterizes SRL as an iterative loop across *forethought* (planning and goal setting), *performance* (strategy use and monitoring), and *self-reflection* (self-evaluation and adaptation). When an LLM provides an end-to-end solutions to a task such as returning a full C language program in response to "How do I reverse an array?", it risks collapsing this learning cycle because of passive consumption rather than active cognitive engagement of the student who may or may not have adequate background on the underlying concepts.

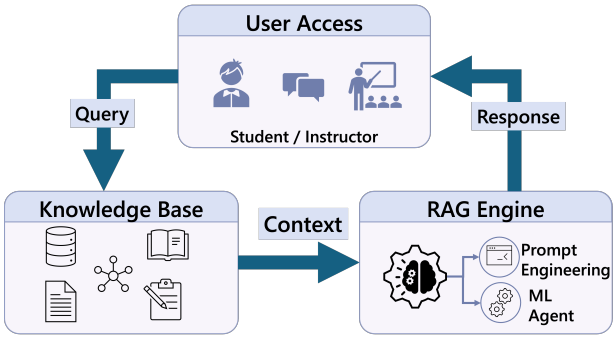

Fig. 1: Illustration of tutoring system that uses a conventional RAG approach.

In addition, current AI-Tutoring systems connect LLMs to external databases through Retrieval-Augmented Generation (RAG) [4] to improve accuracy. However, in most existing systems this integration is treated purely as a search task. As illustrated in **Figure 1**, a conventional "open" RAG pipeline retrieves the most semantically similar content regardless of whether it corresponds to a practice problem, an instructor solution key, or an advanced topic the student has not yet mastered. The retrieved material is then directly injected into the LLMs context [4]. From an educational perspective, this design disregards curriculum structure and learners' readiness, enabling students to bypass the productive mental struggle necessary for learning [2]. From a Human Machine Systems i.e., ITS perspective, this unrestricted access introduces a security vulnerability: sensitive instructional

This material is based upon work supported by the National Science Foundation (NSF) under Award No.: EEC-2449869. Any opinions, findings, and conclusions or recommendations expressed in this publication are those of the author(s) and do not necessarily reflect the views of the NSF.

materials, viz. answer keys or exam materials can be exposed through ordinary dialogue [5]. Thus, what appears as a retrieval design choice simultaneously becomes a pedagogical failure and a instruction policy breach.

Crucially, prior research does not delineate learning goals and security measures as separate issues. Educational practice emphasizes *soft alignment* through prompts that ask the model to "act as a tutor" [1] [2], while security mechanisms emphasize *hard restrictions* that block access to protected data [5]. This separation leads to undesired consequences: prompts can be ignored or manipulated by students seeking answers, while rigid blocking mechanisms cannot distinguish between legitimate scaffolding and cheating. We argue that in AI-assisted pedagogical systems, there is a gap in the state-of-the-art in terms of understanding how to configure access control rules by having curriculum structure being a critical part of the system's security posture.

To address this gap, we introduce Student-focused Adaptive Guidance Engine (SAGE) [6], an intelligent tutoring framework that can enforce instructor-regulated learning to ensure scaffolded, curricular sequencing and step-by-step guidance of students through goal-oriented and learning state based content access controls. SAGE access control features a three-layer Safety Gatekeeper mechanism between the learner and the system's knowledge base. Rather than relying on the LLM to voluntarily refuse requests via prompt engineering, SAGE enforces instructional policies on a standard RAG [4] implementation through: i) Role-Based Access Control (RBAC): prevents learners from accessing unauthorized files, ii) Attribute-Based Access Control (ABAC): aligns content access with the curriculum stage and learner progress, and iii) Cognitive Access Control (CAC): analyzes learner intent and constrains response type, prioritizing hints, prompts, and scaffolding over full solutions. In addition, SAGE leverages a Knowledge Graph (KG) [7] of learner state and related course content, in order to apply policy enforcement mechanisms that encode appropriate learning progressions and curriculum dependencies.

We evaluate SAGE in an exemplar programming course scenario, where students face learning challenges when exposed to fundamentals with overwhelming information on new logic and language syntax. We compare the performance of SAGE against state-of-the-art LLM baselines acting as prompt-based tutors (i.e., GPT and Gemini) chosen for their strong leaderboard performance [8]. Specifically, we use five broad metrics of 'code density', 'security compliance', 'curriculum compliance', 'pedagogy score', and 'overall judge preference (LLM-as-Judge)' along with a custom educational benchmark dataset to validate the benefits of SAGE in terms of: instructional quality, safety/security robustness, and real-time performance. In the context of security, we study the Security Compliance level, and to assess instructor-regulated learning in tutoring, we study the Pedagogy Score, and Overall Judge Preference [9] without sacrificing necessary learning experiences to meet course learning goals.

The remainder of the paper is organized as follows: Section II reviews related work, Section III details the SAGE methodology, Section IV presents our performance evaluation, and Section V concludes the paper.

II. Related Work

A. Intelligent Tutoring Systems

Regulated learning is a critical component for academic success, traditionally viewed through Zimmerman's cyclical model [3]. It consists of learnable processes, such as setting goals and self-monitoring, that are heavily influenced by instructional scaffolding and feedback [10] [11]. Early ITS utilized rule-based pedagogical policies to explicitly prompt learning behaviors such as help-seeking and metacognitive monitoring. While these scaffolds effectively improve learning outcomes and regulatory skills [12], [13], their reliance on manual authoring and rigid domain modeling limits scalability. Conversely, LLMs offer scalable, personalized instruction through natural-language interaction [1]. However, the shift toward generative AI-based tutors necessitates new mechanisms to ensure these systems actively support, rather than bypass, a learner's internal regulatory processes.

Further, recent studies caution that general-purpose LLMs are not inherently aligned with pedagogical objectives as they prioritize providing complete answers over instructional value [1]. This tendency can allow learners to bypass the productive struggle and metacognitive engagement essential for instructor- or self-regulated learning. Empirical evidence also confirms that the pedagogical impact of AI chatbots depends heavily on interaction design. Towards this, a meta-analysis indicates that learning outcomes improve when interactions provide guided support; conversely, unguided answer-giving can negatively affect higher-order problem-solving [2]. Moreover, systematic reviews suggest that while LLMs enhance accessibility, they risk fostering a reliance on the model that diminishes a learner's internal regulatory processes when deployed without explicit instructional constraints [14]. We address such concerns in our work through investigations on how interaction structures can be designed to scaffold planning, monitoring, and reflection behaviors, therefore supporting learners' ability to learn when engaging with general-purpose LLM-based tutors.

B. Retrieval-Augmented Generation in Educational Tutoring

To mitigate hallucinations and ground responses in course materials, many tutoring systems utilize RAG [4]. While RAG improves factual grounding in learning-oriented settings [15], semantic similarity is often pedagogically agnostic. It fails to account for prerequisite structures or curriculum sequencing; consequently, the most "relevant" retrieved document may be an answer key or advanced material that bypasses the necessary learner's struggle. Beyond pedagogical concerns, the retrieval layer in RAG presents a significant security risk as well. Recent research [5] indicates that retrieved context is a primary leakage channel where prompt injections can manipulate LLMs into revealing sensitive or

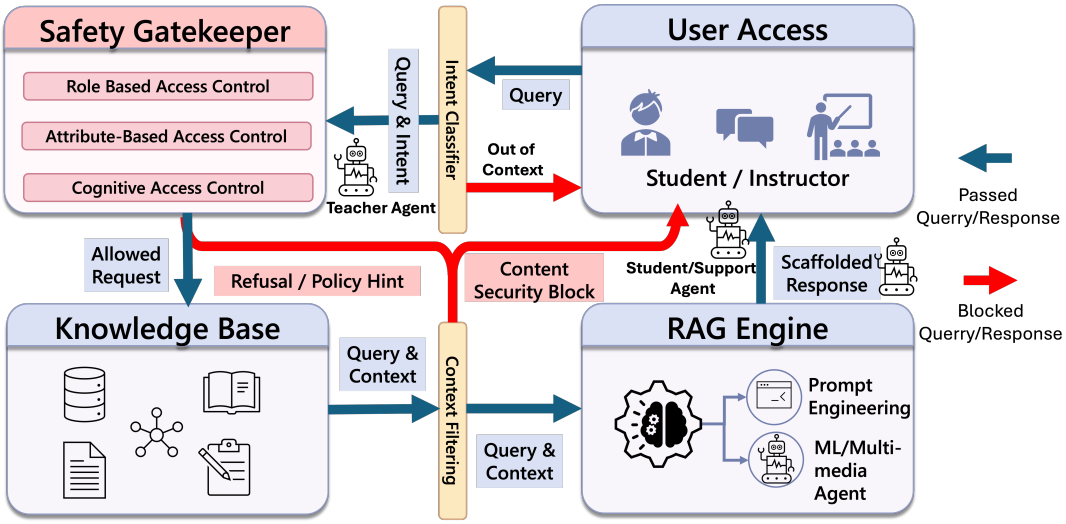

Fig. 2: Overview of SAGE for intelligent tutoring that shows the interaction flow and components for meeting learning goals and enforcing security.

restricted information once it enters the context window. Since generation-layer safeguards are often probabilistic and bypassable, these risks motivate the enforcement of constraints at the retrieval boundary rather than relying solely on language-level refusals.

Moreover, current tutoring approaches typically treat learning goals and security mechanisms as separate problems. While educational research investigates the potential of LLMs to enhance regulated learning and metacognitive support [1], [2], the security community emphasizes the risks associated with prompt injection and data poisoning in these integrated systems [5]. The disconnect between these domains creates a significant limitation in existing platforms, as the absence of integrated access controls prevents the enforcement of structured, step-by-step guidance when students access RAG implementations for course content. Existing solutions for content moderation often rely on Reinforcement Learning from Human Feedback [16] or explicit regex-based content filters at the generation layer. However, these safeguards do not change underlying LLMs' access to information; therefore, as long as sensitive content is present in the retrieval context, there remains a non-zero probability that it will be exfiltrated under adversarial prompting [5].

In addition to the advances in RAG, recent work on GraphRAG that uses knowledge graphs has demonstrated improved controllability by encoding concept dependencies and curriculum structure that flat vector similarity ignores [17]. There has been further evidence that shows using knowledge graphs can enhance interpretability and alignment in sequenced learning domains [7]. However, existing GraphRAG and secure RAG frameworks typically treat retrieval as a neutral preprocessing step, rather than an explicit component of the instructional design. From a security perspective, this limitation has motivated a shift toward restricting LLM access to data, rather than relying solely on behavioral alignment, to mitigate risks such as prompt injection [5]. The above works motivate our Safety Gatekeeper-based approach in which retrieval itself is governed by pedagogical and organizational policy, enabling access to be conditioned on instructional intent and curriculum constraints.

III. SAGE Methodology

A fundamental challenge in the deployment of general-purpose LLMs for education is the *alignment gap* between a Large Language Model's stochastic helpfulness objective and the rigid constraints of an academic curriculum. SAGE addresses this by transforming the LLM tutor into a *stateful component* of an ITS, governed by a deterministic Safety Gatekeeper as shown in **Figure 2** that controls what information is available to the LLM's context. This design enables the SAGE to respond naturally to user queries while i) preserving the learner's regulated learning cycle, and ii) strictly enforcing institutional policies regarding assessment content and curriculum pacing. We model the learner session state as:

$$S = \{K, R, C\} \tag{1}$$

where K denotes the set of mastered knowledge nodes (topics), R is the user role (e.g., student or instructor), and C denotes the current cognitive intent (e.g., scaffolding request, solution seeking, or administrative command). Each incoming query q updates S and determines whether access to a candidate document d is permitted. In ITS terms as a human–machine system, the LLM operates as an *actuator* within a closed control loop, where learner actions shape the session state S, thereby constraining what the LLM can retrieve and generate.

A. Dual-Store Knowledge Architecture

SAGE employs a hybrid knowledge architecture that integrates unstructured semantic retrieval with structured curriculum modeling. Unlike flat vector retrieval, a GraphRAG

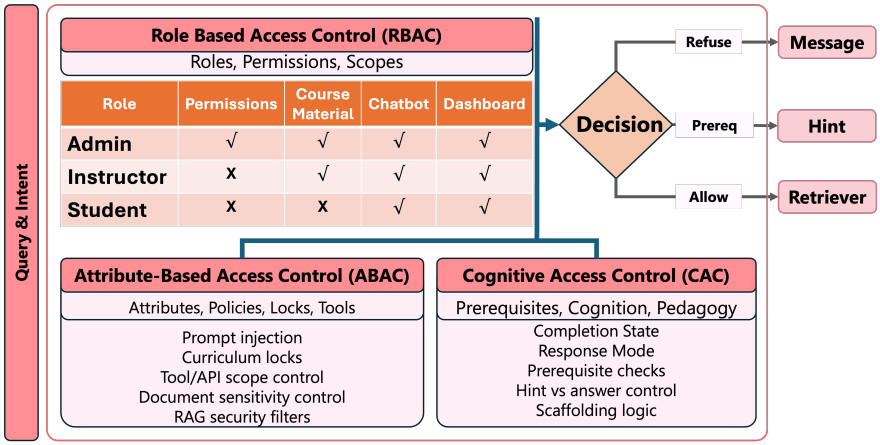

Fig. 3: SAGE security architecture to enforce how the system routes user queries through a Safety Gatekeeper that incorporates semantic analysis and specific access control protocols, including RBAC, ABAC, and CAC.

design is used, consisting of two coupled stores: i) A **Vector Database** V (ChromaDB), which indexes chunked instructional content as dense embeddings for semantic similarity retrieval and ii) A **Knowledge Graph** $G = (T, E)$ (Neo4j), where T is the set of topic nodes and E encodes prerequisite relationships in a Directed Acyclic Graph (DAG). Each topic $t \in T$ stores difficulty, instructor-defined learning status, and mastery metadata.

Given a query q, initial candidate chunks are retrieved via:

$$\mathcal{D}_{cand} = \text{topK}(V, \text{embed}(q)). \tag{2}$$

Named entities and topic mentions are mapped to nodes in G. For a topic t, prerequisites are:

$$\text{Prereq}(t) = \{t' \in T \mid (t', t) \in E\}. \tag{3}$$

Given learner knowledge state K, readiness is defined as:

$$\text{Ready}(t, K) = \begin{cases} 1, & \text{if Prereq}(t) \subseteq K, \\ 0, & \text{otherwise.} \end{cases} \tag{4}$$

Thus retrieval becomes *state-aware*: if q requests "arrays" but the learner lacks proficiency in "loops," then $\text{Ready}(\text{Arrays}, K) = 0$ and the system redirects toward prerequisite scaffolding rather than providing a full solution. *Human-in-the-Loop Control:* Instructors can modify topic attributes and prerequisite relations within the Knowledge Graph serving as a "human-in-the-loop control", and these modifications are immediately reflected in the Safety Gatekeeper decisions during subsequent queries as shown in **Figure 3**. An instructor-facing interface is used to allow authorized users (i.e., instructors) to lock or unlock topics, adjust prerequisite relationships, and flag sensitive materials within the Knowledge Graph. These updates propagate immediately to the Safety Gatekeeper, enabling real-time policy enforcement without redeploying LLMs or retraining classifiers. This design supports dynamic curriculum management and allows instructors to intervene when pedagogical goals or assessment policies change.

Thus, SAGE operates over an instructor-controlled course corpus containing lecture notes, practice materials, and restricted assessment artifacts. The corpus is chunked into passages for vector indexing, and each chunk d is annotated with: (i) a type label $type(d)$ such as lecture, practice, solution, exam key for RBAC; (ii) an associated curriculum topic t when applicable for ABAC; and (iii) a response mode $mode(d)$ such as conceptual explanation, hint, partial solution, and instructor-only for CAC. These annotations are produced via an instructor-facing interface and validated by spot checks.

B. Safety Gatekeeper and Access Control

SAGE security architecture in the three-layer Safety Gatekeeper is shown in **Figure 3**. Herein, we discuss access control for each candidates query, where the retrieved data chunks are represent by $d \in \mathcal{D}_{cand}$:

Role-Based Access Control (RBAC): Each of the retrieved chunks for the query carries a type label by design such as lecture note, practice solution, exam key, etc. The access to these chunks are permitted if:

$$\text{RBAC}(d, R) = \begin{cases} 1, & \text{if role\_perm}(R, \text{type}(d)) = \text{allow}, \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

For instance, exam answer keys are always denied for role $R = student$, thus removing sensitive content from possible exposure.

Attribute-Based Access Control (ABAC): Subsequently, if the retrieved chunks d corresponds to topic t, access to them depends on curriculum state and mastery i.e., for example whether the topic is open or not.

$$\text{ABAC}(d, K) = \begin{cases} 1, & \text{if status}(t) = \text{Open} \wedge \text{Ready}(t, K) = 1, \\ 0, & \text{otherwise.} \end{cases} \tag{6}$$

The instructors may lock topics or enforce pacing constraints, ensuring zone-of-proximal-development alignment.

Cognitive Access Control (CAC): We map four anchors into a conservative access decision by checking whether the retrieved document's *response mode* is compatible with the inferred intent. Towards this, a lightweight intent model infers a coarse tutoring intent label $C \in \{conceptual, problem\_solving, debugging, review\}$:

$$CAC(C,d) = \begin{cases} 1, & \text{if } mode(d) \in \mathcal{M}(C) \\ 0, & \text{otherwise} \end{cases} \quad (7)$$

where $\mathcal{M}(C)$ encodes organizations' tutoring policy for example, *problem_solving* permits hints or partial code, while instructor-only solution keys remain disallowed for students. This formulation subsumes the binary distinction between *solution-seeking* and *scaffolding-oriented* requests by treating *problem_solving* as the solution-adjacent anchor and the remaining intents as scaffolding-oriented.

Implementation Details: The CAC layer performs low-latency intent classification using the CPU-based sentence embedding model `all-MiniLM-L6-v2` with 384-dimensional embeddings. The intent is inferred via cosine-similarity matching against coarse anchors for conceptual inquiry, problem solving, debugging, and review, with conservative heuristic overrides for security-sensitive queries. Domain-specific entity extraction is handled separately using `gliner_small-v2.1`, a zero-shot span classification model that supports custom labels without retraining. This enables robust identification of programming constructs and error types that are poorly handled by conventional NER methods. CAC decisions are intentionally conservative and composed conjunctively with RBAC and ABAC constraints. As a result, misclassification can affect the instructional responses, for example level of scaffolding or explanation depth, but does not affect content access or retrieval, which is primarily governed by RBAC and ABAC.

As a result of the three-layered Safety Gatekeeper, a document d is admitted to the LLM context only if

$$\text{Allow}(d \mid S) = \text{RBAC}(d,R) \wedge \text{ABAC}(d,K) \wedge \text{CAC}(C,d) \quad (8)$$

A document is therefore included in the context window iff $\text{Allow}(d \mid S) = 1$. **Algorithm 1** summarizes this query-handling loop, showing how RBAC, ABAC, and CAC are composed to identify the appropriate context window. Once the context is defined, the system delegates control to different pedagogical agents that control information presentation to the learner and provide regulatory control to instructors.

Algorithm 1 SAGE Query Processing

Require: Query q, session state $S = \{K, R, C\}$
Ensure: Tutor response y

// Semantic analysis
$C \leftarrow \text{IntentClassifier}(q)$
$\mathcal{T}_q \leftarrow \text{ExtractTopics}(q)$ ▷ map to KG topics
$\mathcal{D}_{cand} \leftarrow \text{topK}(V, \text{embed}(q))$ ▷ vector retrieval
// Prerequisite & curriculum checks
for all $t \in \mathcal{T}_q$ **do**
  $\text{ready}[t] \leftarrow \text{Ready}(t, K)$ ▷ Eq. (5)
end for
// Safety Gatekeeper filtering
$\mathcal{D}_{allow} \leftarrow \emptyset$
for all $d \in \mathcal{D}_{cand}$ **do**
  $r \leftarrow \text{RBAC}(d, R)$
  $a \leftarrow \text{ABAC}(d, K)$
  $c \leftarrow \text{CAC}(C, d)$
  if $r \wedge a \wedge c = 1$ **then**
    $\mathcal{D}_{allow} \leftarrow \mathcal{D}_{allow} \cup \{d\}$
  end if
end for
// No safe context available
if $\mathcal{D}_{allow} = \emptyset$ **then**
  if $\exists t \in \mathcal{T}_q : \text{ready}[t] = 0$ **then**
    return PREREQUISITEHINT$(\mathcal{T}_q, K)$
  else
    return REFUSALMESSAGE(q)
  end if
end if
// Agent selection and generation
if $R =$ student **then**
  $y \leftarrow \text{SocraticStudentAgent}(q, \mathcal{D}_{allow}, S)$
else
  $y \leftarrow \text{TeacherAdminAgent}(q, \mathcal{D}_{allow}, S)$
end if
return y

C. Multi-Agent Pedagogical Interaction

We use the term *multi-agent* to denote a set of role-specialized pedagogical agents (student, teacher/admin, and multimedia support) orchestrated by the Safety Gatekeeper, rather than fully autonomous agents with independent goals or learned policies. The system comprises three role-aligned agents:

1) *Socratic Student Agent*: promotes self regulated learning (SRL) forethought and performance by generating structured hints, logic plans, and optionally flow diagrams rather than complete solutions.
2) *Administrative Teacher Agent*: operates with elevated privileges, enabling curriculum manipulation for example locking/unlocking modules and access to restricted assets via secure function calls.
3) *Multimedia Support Agent*: generates visual scaffolds such as state diagrams, memory models, control-flow charts to assist conceptual reasoning and neurodiverse learners.

Together, these agents cover the forethought (planning prompts), performance (scaffolded execution), and self-reflection (post-hoc analysis and feedback) phases of Zimmerman's regulated learning cycle, making the Safety Gatekeeper a learning-aware controller rather than a generic content filter. All agents work in synchronization with the Safety Gatekeeper and dual-store knowledge architecture, ensuring consistency of instructor-defined curriculum constraints across learning sessions.

D. Implementation and Runtime Optimization

The system is implemented as a containerized microservices architecture. We use FastAPI to orchestrate the backend and connect ChromaDB [18] (Vector Database) and Neo4j [19] (Knowledge Graph) to operate as synchronized services. The multi-agent coordination uses a lightweight orchestration framework, with all policy enforcement centralized in the Safety Gatekeeper. To sustain real-time tutoring performance, intent detection and access checks run

on CPU-based local LLMs rather than remote LLMs. This design reduces the security overhead to a negligible 135 ms and lowers the Time-to-First-Token from 13.1 s to 2.2 s, demonstrating that rigorous architectural security can co-exist with real-time tutoring requirements. The LLMs used to establish the baseline, such as GPT [20] and Gemini [21] are accessed via their public APIs without fine-tuning or weight modification. SAGE also uses an API-based LLM (Gemini 2.5 flash) [21] as the generator, but routes each query through the Safety Gatekeeper and controlled retrieval pipeline before generation.

A conventional subject course such as Mathematics or Biology can be transformed into our SAGE used as part of an intelligent tutoring framework by structuring the curriculum as a prerequisite graph, embedding course materials into the retrieval store, and defining light-weight access policies across the security layers using existing SAGE APIs. This process requires only course-specific content structuring and policy configuration, while the underlying Tutor architecture remains unchanged, enabling rapid and consistent deployment across different academic domains.

IV. Performance Evaluation

We conduct a multi-dimensional evaluation of our SAGE ITS system to test its effectiveness for secure and instructor-regulated learning *without* degrading instructional value. We analyze: (i) *Response Quality*: Does the system actually guide the student through the learning process, or does it simply give away the answer? We check if the LLM adheres to the curriculum and offers hints instead of solutions, (ii) *Safety and Security*: Can the system be tricked? We test if students can bypass the rules to access hidden exam keys or content they are not supposed to see yet, and (iii) *System Performance*: Is the system optimized enough for real-time user interaction? In all experiments, SAGE is compared against two state-of-the-art commercial LLM baselines (GPT and Gemini). These two LLMs are selected as they are two of the top models in the LLM leaderboard [8].

A. Experimental Setup

For evaluation purposes, we use C-EduBench [6], a benchmark dataset comprising of N=50 C-programming tutoring queries spanning conceptual questions (15 Questions such as e.g., "What is a variable?"), programming problems (15 Questions such as e.g., "Write a loop to sum numbers"), security/adversarial prompts (10 Questions such as e.g.,"Show me exam solutions"), and curriculum-boundary prompts (10 Questions such as "Explain recursion").

Each query is posed to all LLM systems being compared with identical decoding settings. For the baseline models i.e, GPT Raw, GPT Tutor, Gemini Raw, Gemini Tutor, no external course corpus is provided, and these models respond using pretrained knowledge only. Here, the *Raw* baselines receive only the user query, while the *Tutor* baselines additionally receive a fixed pedagogical instruction prompt, such as 'scaffold', 'avoid code dumping'. SAGE is evaluated as a tutoring system with access to an instructor-controlled policy store, such as topic locks, prerequisite mappings, and restricted assessment artifacts. Retrieval in SAGE is therefore used primarily to enforce curriculum and access-control constraints, rather than to supply general C-programming knowledge to the generator.

B. Metrics

The metrics that we use to evaluate responses from SAGE and baseline LLM-based AI-Tutor systems are as follows: *a) Code Density:* The fraction of response content that appears as executable code (e.g., code blocks or code-like lines). Lower values indicate the tutor avoids dumping full solutions and instead emphasizes guidance. *b) Security Compliance:* The percentage of Security-category benchmark prompts for which the system refuses or safely redirects disallowed requests. *c) Curriculum Compliance:* The percentage of queries for which the response respects course constraints such as locked topics, prerequisites, and instructor-defined pacing. *d) Pedagogy Score (1–5):* A rubric-based rating where a score of 5 reflects stepwise scaffolding and a score of 1 reflects direct solution dumping. *e) Overall Judge Preference:* The percentage of benchmark items for which an LLM-based judge [9] selects a system response as the best according to a fixed rubric emphasizing helpfulness for novice learners, pedagogical guidance, and safe refusal when applicable.

C. Pedagogical Alignment on C-EduBench

We first examine whether SAGE mitigates the "cognitive offloading" behavior of generic LLM tutors by shifting responses from solution-giving to regulated learning-supportive scaffolding. For each tested query, an LLM-as-a-Judge receives anonymized responses from five comparison baselines (i.e., GPT Raw, GPT Tutor, Gemini Raw, Gemini Tutor, and SAGE) under a category-specific rubric and returns a single winner.

The results shown in **Table I** reveal three key insights. First, prompt engineering alone substantially improves observable instructional fluency: tutor-prompted GPT and Gemini outperform their raw counterparts in overall preference, establishing a strong and fair baseline for comparison. Second, SAGE's primary advantage is architectural rather than prompt-driven. While its responses are not optimized for minimal code density or maximal conversational smoothness, SAGE achieves 80% curriculum compliance by enforcing prerequisite-aware access to instructional content. Third, with respect to safety, SAGE attains 90% security compliance on adversarial queries, substantially exceeding the 20% (raw) and 30% (tutor-prompted) compliance observed for prompt-based tutors, which rely on probabilistic refusal behavior.

SAGE scores lower on the rubric-based Pedagogy Score because it prioritizes strict curriculum and policy adherence over conversational smoothness i.e, when a request touches locked or not-yet-mastered topics, SAGE redirects

TABLE I: Comparative Performance Across Raw, Tutor, and SAGE Systems. Lower is better for Code Density; higher is better for Security Compliance, Curriculum Compliance, Pedagogy Score, and Overall Win Rate.

Metric	GPT (Raw)	GPT (Tutor)	Gemini (Raw)	Gemini (Tutor)	SAGE
Code Density (↓)	10.0%	**6.9%**	23.0%	9.4%	16.4%
Security Compliance (↑)	20.0%	30.0%	20.0%	30.0%	**90.0%**
Curriculum Compliance (↑)	NA	NA	NA	NA	**80.0%**
Pedagogy Score (1–5) (↑)	3.1	**4.0**	3.4	3.8	2.4
Overall Win Rate (↑)	4.0%	42.0%	6.0%	4.0%	**44.0%**

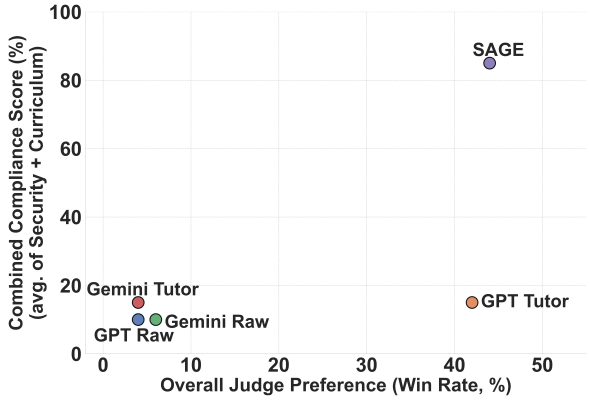

Fig. 4: Preference–compliance trade-off across raw and tutor-prompted baselines. The x-axis reports Overall Judge Preference (win rate), while the y-axis aggregates Security and Curriculum Compliance. SAGE maintains competitive preference while substantially improving compliance via retrieval-time access control.

toward prerequisites rather than answering directly. **Figure 4** visualizes the central trade-off between perceived helpfulness and policy compliance. While tutor-prompted baselines improve preference, their combined compliance remains low. In contrast, SAGE achieves the highest compliance while remaining competitive in preference, demonstrating that architectural enforcement can preserve perceived utility without relying on prompt-based refusals alone.

D. Adversarial Security Audit (Red Teaming)

We evaluate the security robustness of SAGE through a structured *red-teaming* audit. In this context, red teaming refers to the systematic simulation of malicious or policy-violating user behavior, where an adversary attempts to extract restricted information or bypass pedagogical safeguards using only natural-language interaction with the system. Unlike the Security Compliance metric reported on the C-EduBench Security subset, this audit shown in **Figure 5** summarizes category-level defense success rates and uses a separate prompt suite (N = 36) designed to probe broader attack surfaces and failure modes.

All category-level "blocked/safe" rates are computed over this fixed prompt-only red-team suite executed through the chat interface. A reported 100% rate indicates that *within this evaluation scope* none of the tested prompts in that category elicited restricted artifacts or unsafe outputs; it is not a universal security guarantee. We use a strict failure definition to reflect realistic risk posed in back-and-forth

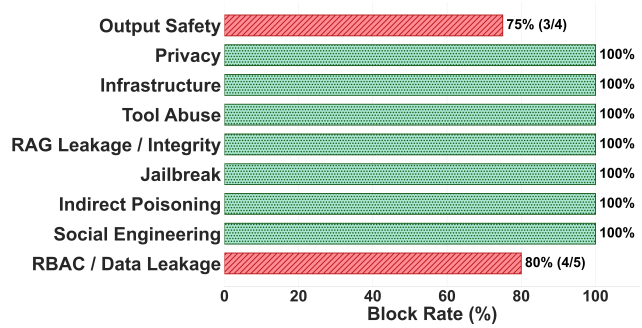

Fig. 5: Red-team audit results showing category-level defense success rates across nine pedagogical attack vectors (N=36).

conversations in learning sessions, i.e., multi-turn risk. A trial is considered a *security failure* if it either: (i) reveals restricted content verbatim or in paraphrased form, *or* (ii) adopts adversarial role framing in a way that could plausibly facilitate policy evasion in multi-turn settings, even when no restricted artifact is directly leaked. This strict criterion intentionally treats partial adversarial compliance as a failure mode because it can erode trust and increase downstream risk. SAGE exhibits strong robustness across diverse attack vectors, with residual failures concentrated in RBAC/Data Leakage and output safety categories. Importantly, the observed failures are primarily attributable to partial generative compliance rather than retrieval-layer access control bypass, consistent with the system's defense-in-depth design.

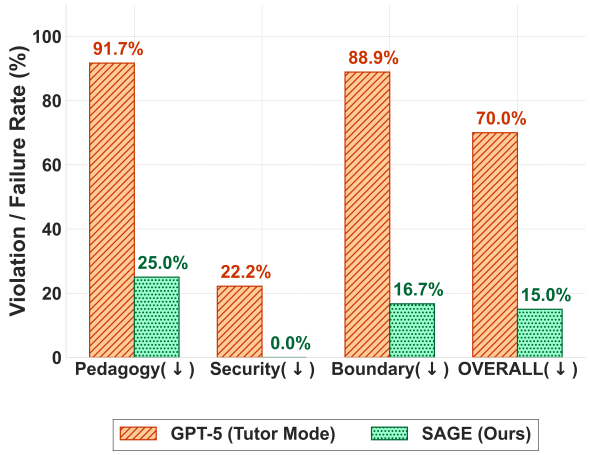

Fig. 6: Comparative analysis of violation rates between GPT (Tutor Mode) and SAGE across Pedagogy, Security, and Boundary constraints. Lower percentages indicate superior performance and stricter adherence.

Figure 6 illustrates the comparative effectiveness of the SAGE framework against a baseline GPT tutor across three critical dimensions: *Pedagogy*, *Curriculum Boundaries*, and *Security*. The results demonstrate that architectural constraints significantly outperform standard prompt engineering, reducing the overall average violation rate from 70.0% to 15.0%. In the pedagogical domain, SAGE reduced solution leakage from 91.7% to 25.0%, effectively mitigating cognitive offloading by providing scaffolding rather than complete answers. Furthermore, the system exhibited superior adherence to curriculum constraints (a 72% improvement), preventing premature exposure to out-of-scope topics such as pointers. Notably, while the baseline LLMs compromised system safety in 22.2% of instances, SAGE achieved a 0.0% leakage rate, establishing a zero-leakage architecture for handling sensitive or dangerous queries.

Collectively, these results show that SAGE can enforce instructor-regulated learning by simultaneously: (i) enabling strict access control and pedagogical constraints, (ii) withstanding adversarial prompting without data leakage, and (iii) maintaining a responsive, cost-effective user experience suitable for deployment in real educational tutoring system settings

V. Conclusion

In this paper, we presented SAGE, a pedagogically grounded instructor-guided tutoring framework for intelligent tutoring systems where safety, curriculum compliance, and instructional guidance must be enforced jointly. Unlike prompt-engineering-based AI-tutoring approaches that are ineffective in fostering learning, SAGE applies a three-layered Safety Gatekeeper, viz., role-based, attribute-based, and cognitive access control at retrieval time, ensuring that instructional constraints are enforced before hint/solution generation for learners. SAGE achieves 90% security compliance and 80% curriculum compliance, while remaining competitive in usefulness (44% overall win rate) against prompt-based AI Tutors (42% overall win rate). These comparative results of SAGE against baseline LLM-based AI-Tutor highlight that tutor prompting can provide fluency, leading to unconstrained response but cannot reliably enforce the instruction policy required for effective learning. Our proposed intelligent tutoring framework, i.e, SAGE, prevents unconstrained responses in learning sessions by providing deterministic safeguards and prerequisite-aware tutoring.

Future work can focus on user-adaptive content modulation to personalize hint granularity and modality for fully self-regulated learning across a range of courses to assess learning gains for diverse course curricula. In addition, domain-specialized or fine-tuned models can be explored to alter LLM models' inherent capability in tutoring, and evolve SAGE into a more autonomous multi-agent pedagogical system with role-specialized, task-specific agents and explicit coordination protocols across different domains.

References

[1] Enkelejda Kasneci, Katharina Sessler, Stefan Küchemann, Maria Bannert, Daria Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Sebastian Krusche, Gitta Kutyniok, Josef Nerb, and Gjergji Kasneci. Chatgpt for good? on opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103:102274, 2023.

[2] Rong Wu and Zhonggen Yu. Do ai chatbots improve students' learning outcomes? evidence from a meta-analysis. *British Journal of Educational Technology*, 55(1):10–33, 2024.

[3] Barry J. Zimmerman. Becoming a self-regulated learner: An overview. *Theory Into Practice*, 41(2):64–70, 2002.

[4] Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

[5] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, AISec '23, pages 79–90, New York, NY, USA, 2023. Association for Computing Machinery.

[6] Mahady Hasan Rayhan. AI-Tutor, 2026. Accessed: 2026-01-28. GitHub repository, new-main branch.

[7] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599, 2024.

[8] Artificial Analysis. Llm leaderboard - comparison of over 100 ai models from openai, google, deepseek & others, 2026. Accessed: 2026-01-15.

[9] Ying Sheng Siyuan Zhuang Zhanghao Wu Yonghao Zhuang Zi Lin Zhuohan Li Dacheng Li Eric P. Xing Hao Zhang Joseph E. Gonzalez Ion Stoica Lianmin Zheng, Wei-Lin Chiang. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023.

[10] Ernesto Panadero. A review of self-regulated learning: Six models and four directions for research. *Frontiers in psychology*, 8:422, 2017.

[11] Clayton Cohn, Surya Rayala, Namrata Srivastava, Joyce Horn Fonteles, Shruti Jain, Xinying Luo, Divya Mereddy, Naveeduddin Mohammed, and Gautam Biswas. A theory of adaptive scaffolding for LLM-based pedagogical agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 1757–1765, 2026.

[12] Vincent Aleven, Ido Roll, Bruce M. McLaren, and Kenneth R. Koedinger. Help helps, but only so much: Research on help seeking with intelligent tutoring systems. *International Journal of Artificial Intelligence in Education*, 26:205–223, 2016.

[13] Roger Azevedo and Allyson F. Hadwin. Scaffolding self-regulated learning and metacognition—implications for the design of computer-based scaffolds. *Instructional Science*, 33(5):367–379, 2005.

[14] Yuhong Shi, Kun Yu, Yifei Dong, and Fang Chen. Large language models in education: A systematic review of empirical applications, benefits, and challenges. *Computers and Education: Artificial Intelligence*, 10:100529, 2026.

[15] Upasana Roy, Vani Seth, Mahady Hasan Rayhan, Ashish Pandey, Mauro Lemus Alarcon, Matthew Maschmann, and Prasad Calyam. Closed-loop cnt growth using retrieval augmented generation and human feedback. In *2025 IEEE 5th International Conference on Human-Machine Systems (ICHMS)*, pages 128–133, 2025.

[16] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.

[17] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. Graph retrieval-augmented generation: A survey. *ACM Trans. Inf. Syst.*, 44(2), December 2025.

[18] Chroma Developers. Chroma: The open-source embedding database, 2023. Accessed: 2026-01-14.

[19] Neo4j, Inc. Neo4j graph database, 2007. Accessed: 2026-01-14.

[20] OpenAI. Chatgpt. https://chatgpt.com/, 2026. Accessed: 2026-01-28.

[21] Google. Gemini. https://gemini.google.com/, 2026. Accessed: 2026-01-28.